

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

40 - Pipelined Datapath

Introduction

- Show single-cycle datapath with pipeline

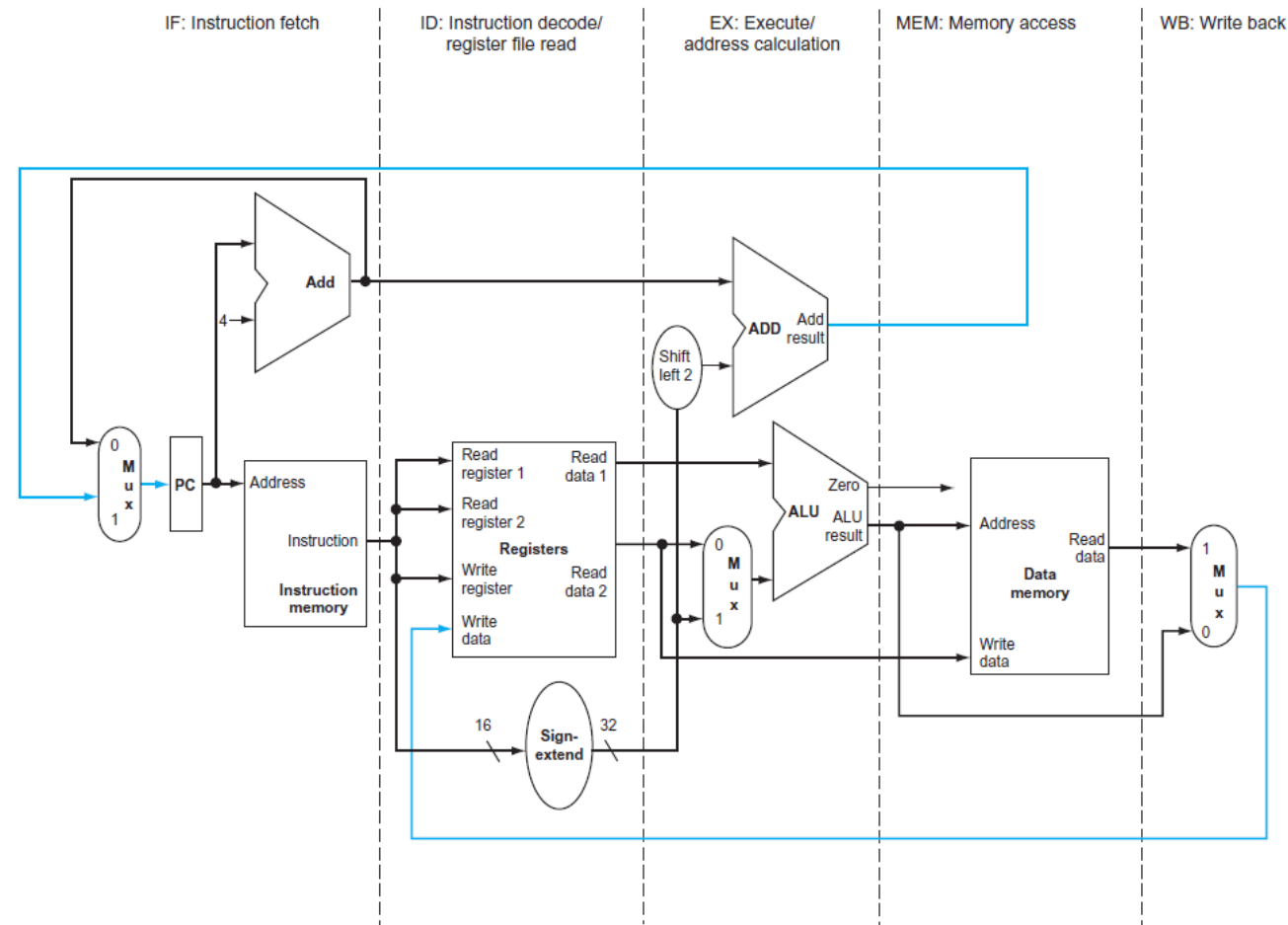
Single-cycle Datapath

- Single-cycle datapath with pipeline stages
- Division of an instruction into five stages means a five-stage pipeline
- Means that up to five instructions will be in execution during any single clock cycle

Pipeline Stages

- Must separate datapath into five pieces, with each piece named corresponding to a stage of instruction execution:
 1. **IF**: Instruction fetch
 2. **ID**: Instruction decode and register file read
 3. **EX**: Execution or address calculation
 4. **MEM**: Data memory access
 5. **WB**: Write back

Pipeline Single-cycle Datapath



Flow Direction

- Instructions and data move generally LTR through the five stages as they complete execution
- Two **exceptions** to LTR flow of instructions
 - WB stage, places result back into register file in middle of datapath
 - Selection of next value of PC, choosing between incremented PC and branch address from MEM stage

Flow Direction

- Data flowing RTL does not affect current instruction
 - Reverse data movements influence only later instructions in pipeline
 - First RTL flow of data can lead to data hazards
 - Second leads to control hazards

Execution Using Single-cycle Datapath

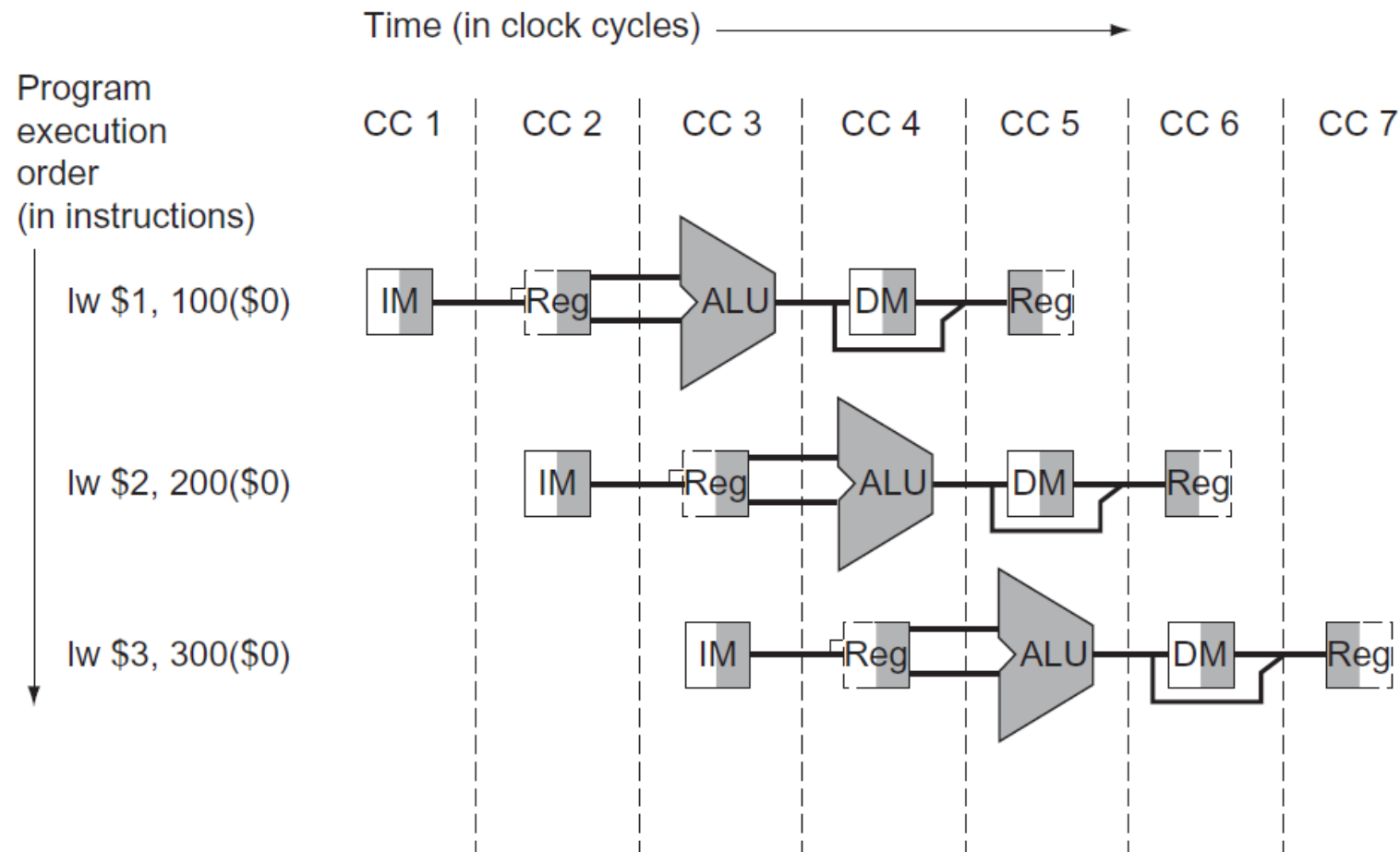
- Figure shows the execution of instructions by displaying their private datapaths on a common timeline

lw \$1, 100(\$0)

lw \$2, 200(\$0)

lw \$3, 300(\$0)

Execution Using Single-cycle Datapath



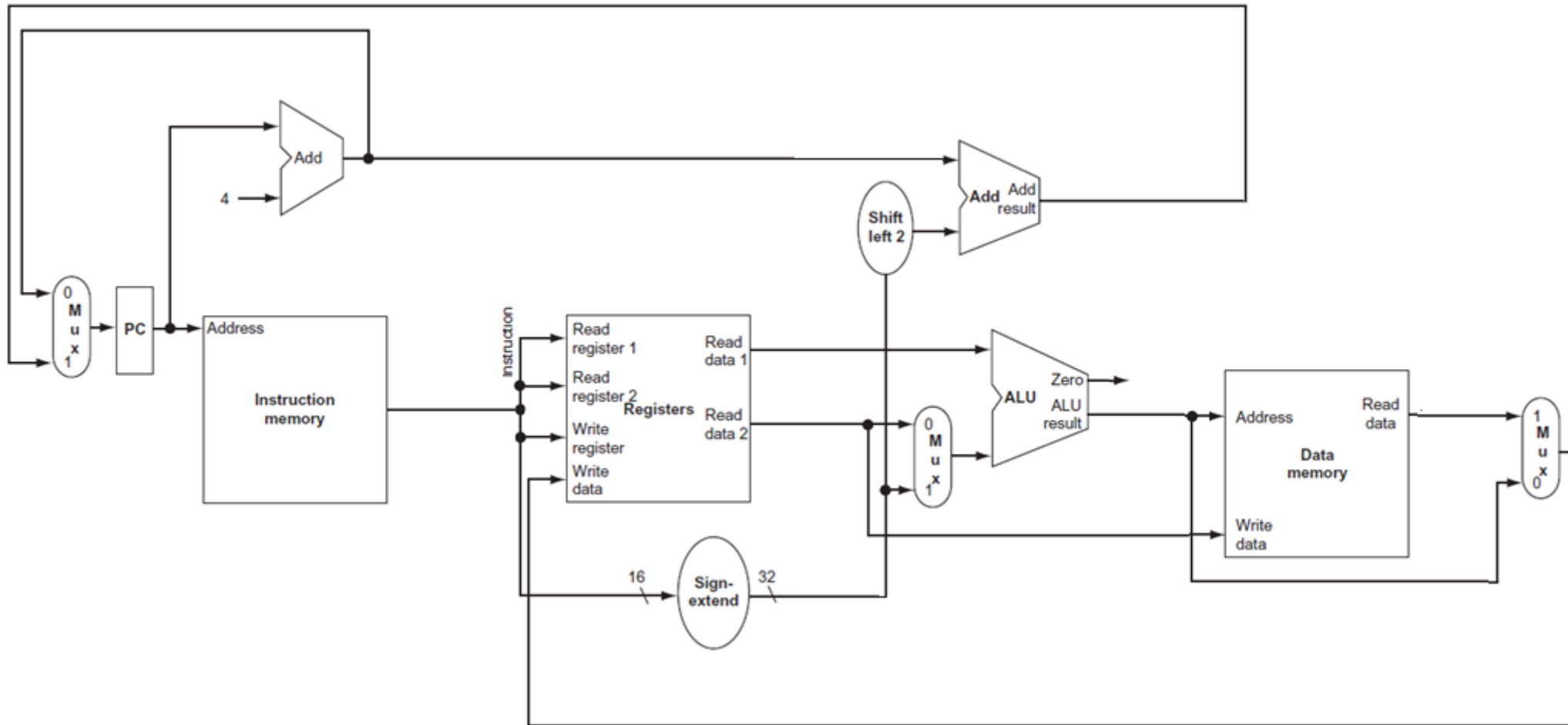
Pipeline Registers

- Figure seems to suggest that three instructions need three datapaths
- Instead, we **add registers** to hold data so that portions of a single datapath can be shared during instruction execution

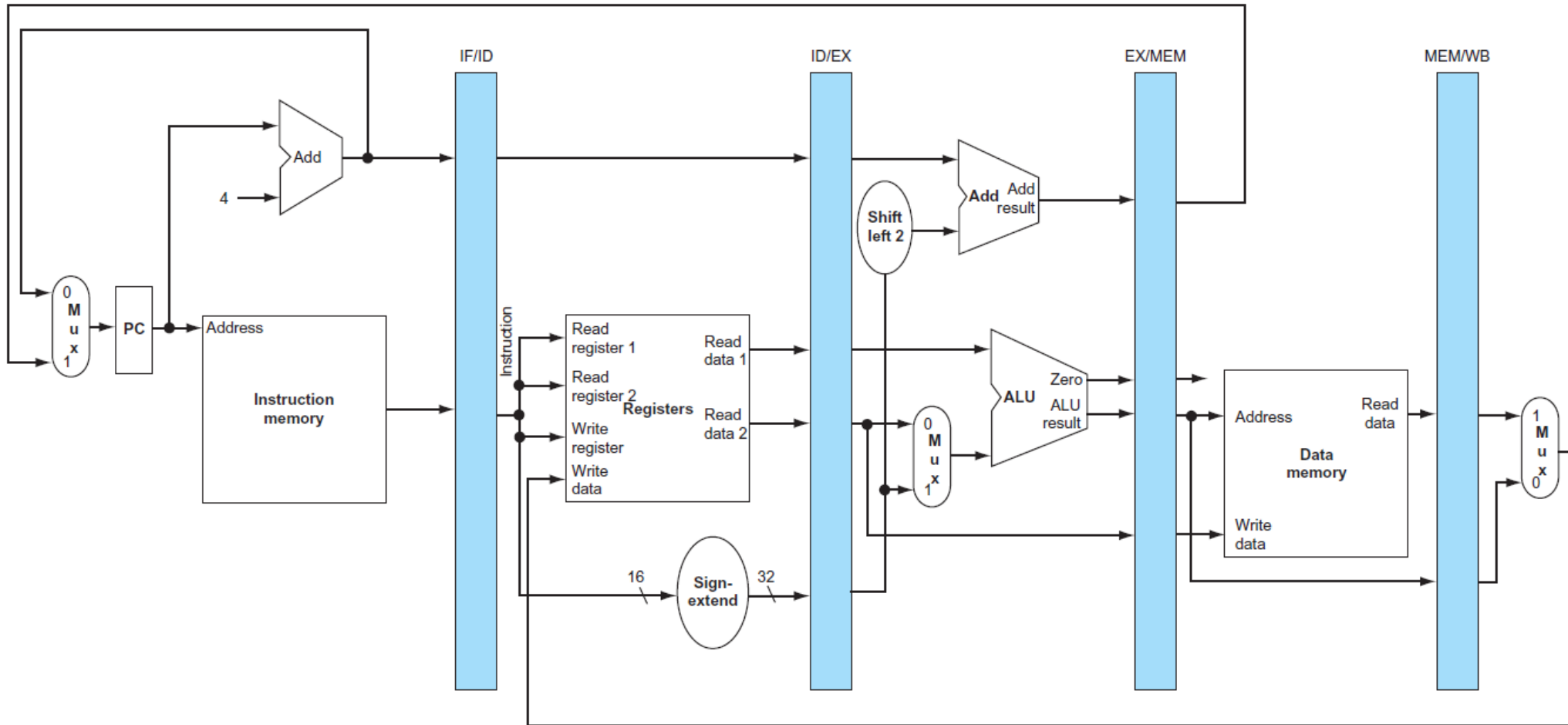
Pipeline Registers

- For example, instruction memory is used during only one of the five stages of an instruction, allowing it to be **shared** by following instructions during the other four stages
 - To retain value of an individual instruction for its other four stages, value read from instruction memory must be **saved in a register**
- Similar arguments apply to every pipeline stage
 - Must place **registers** between stages

Pipeline Registers



Pipeline Registers



Pipeline Registers

- All instructions advance during each clock cycle from one pipeline register to the next
- Registers are named for the two stages separated by that register.
 - For example, pipeline register between IF and ID stages is called IF/ID

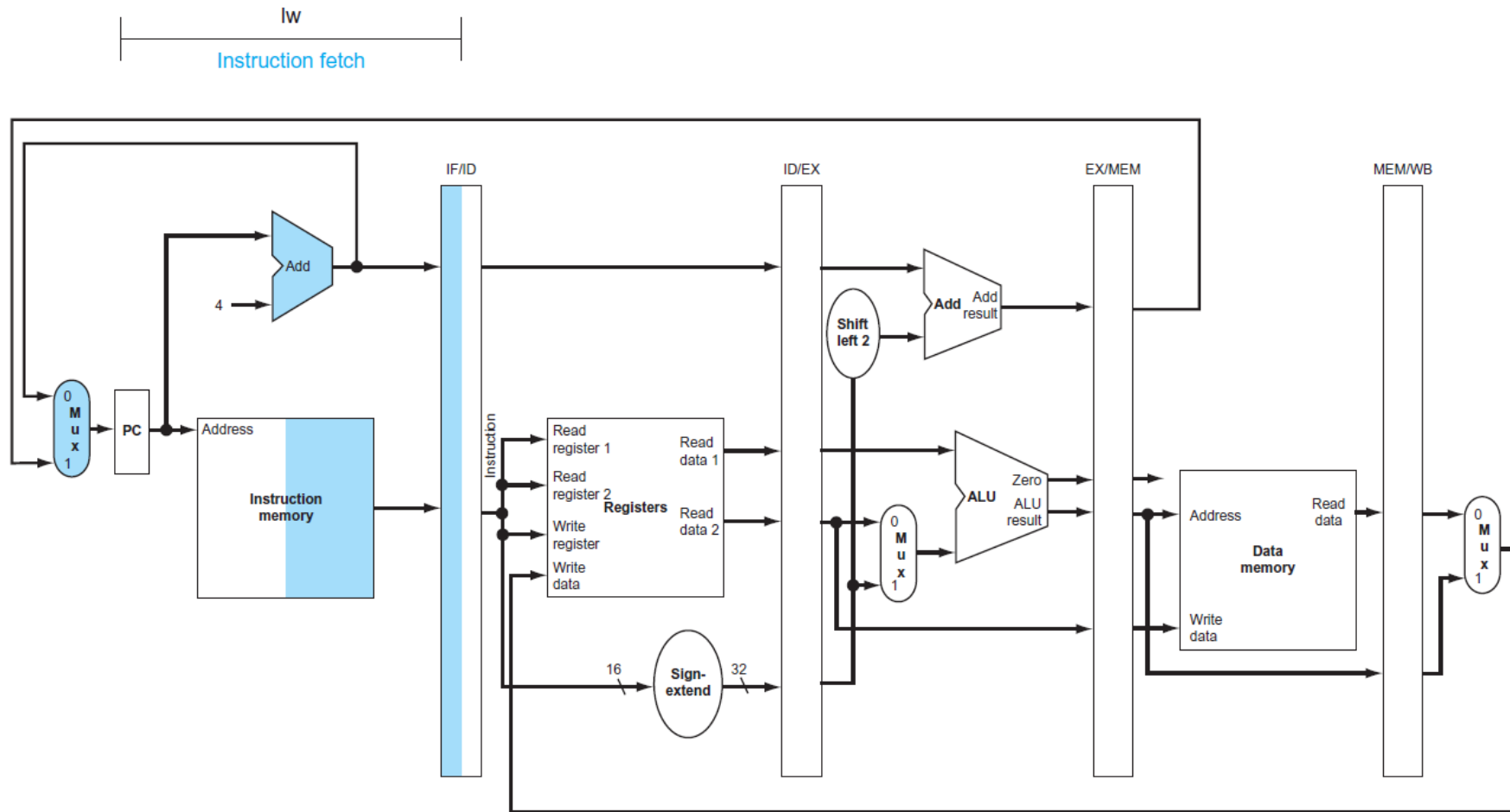
Load-word Instruction Pipeline Datapath

- Load-word Instruction
- Show a load first because it is active in all five stages

Instruction Fetch

- Instruction is read from memory using address in PC and then is placed in IF/ID pipeline register
- Stage occurs before instruction is identified, so top portion of figure works for store as well as load

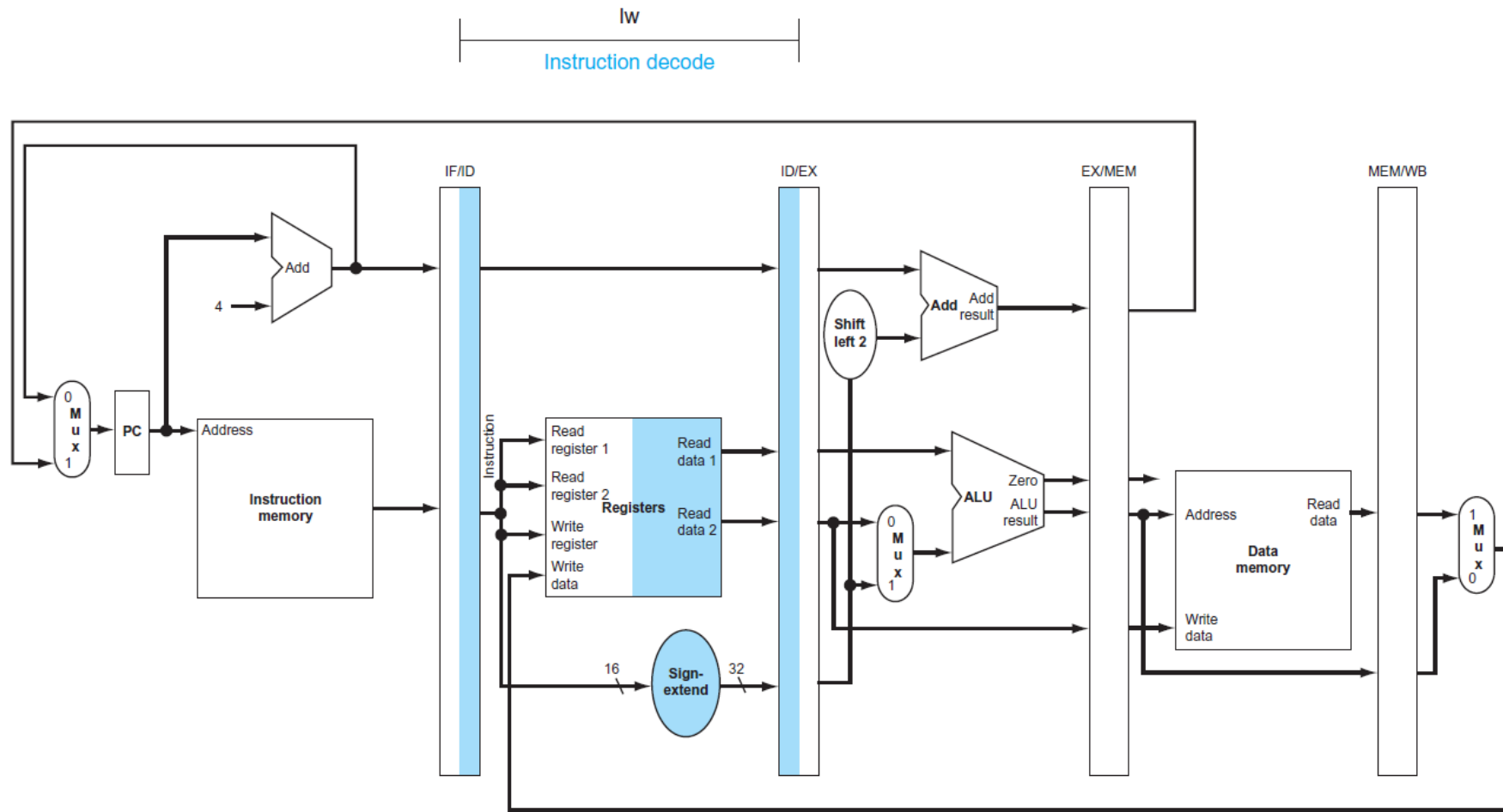
Instruction Fetch



Instruction Decode

- Instruction portion of IF/ID pipeline register supplies:
 - 16-bit immediate field, which is sign-extended to 32 bits
 - Register numbers to read the two registers
- All three values are stored in ID/EX pipeline register, along with the incremented PC address
- Transfer everything that might be needed by any instruction during a later clock cycle

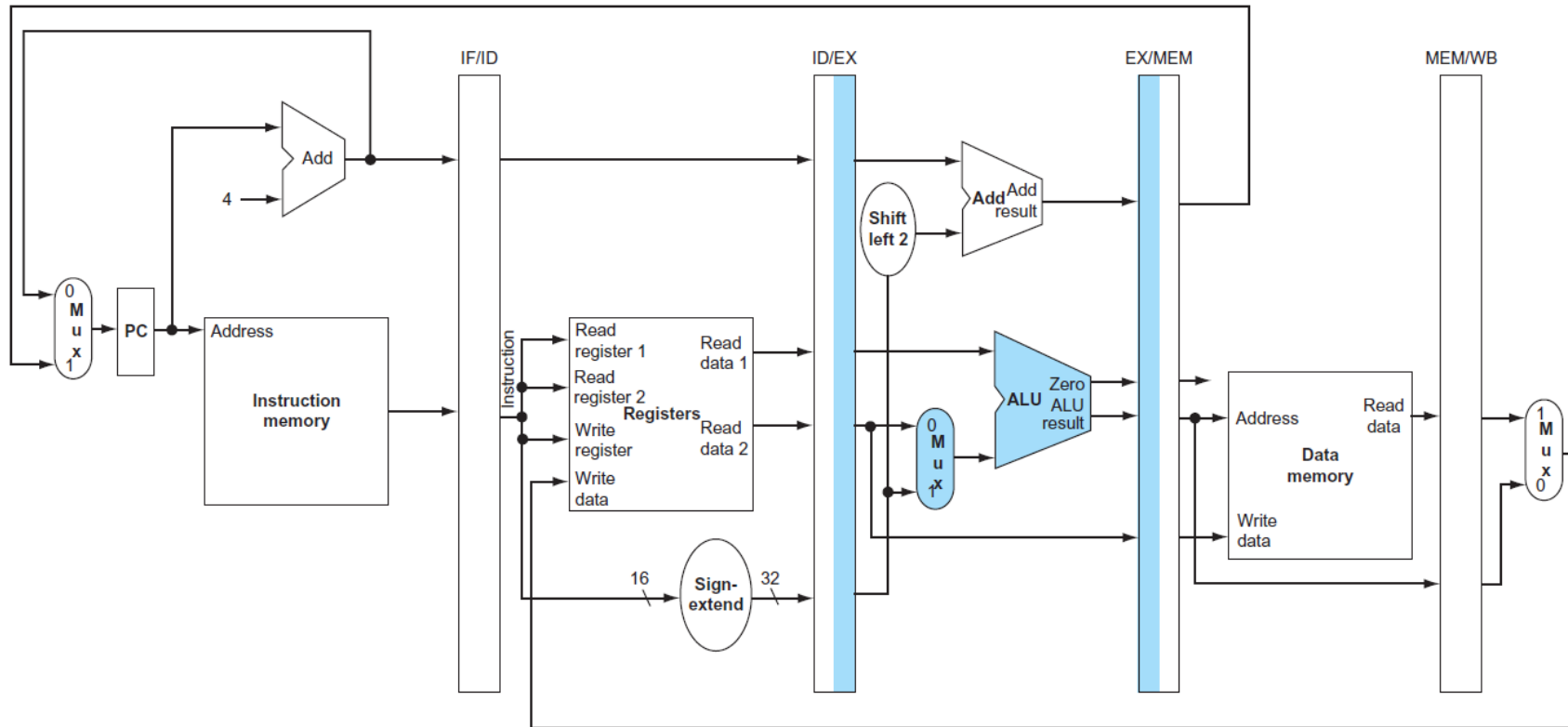
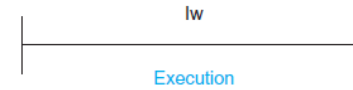
Instruction Decode



Execute

- Read contents of register 1 and sign-extended immediate from ID/EX pipeline register
- Add them using ALU
- Sum is placed in EX/MEM pipeline register

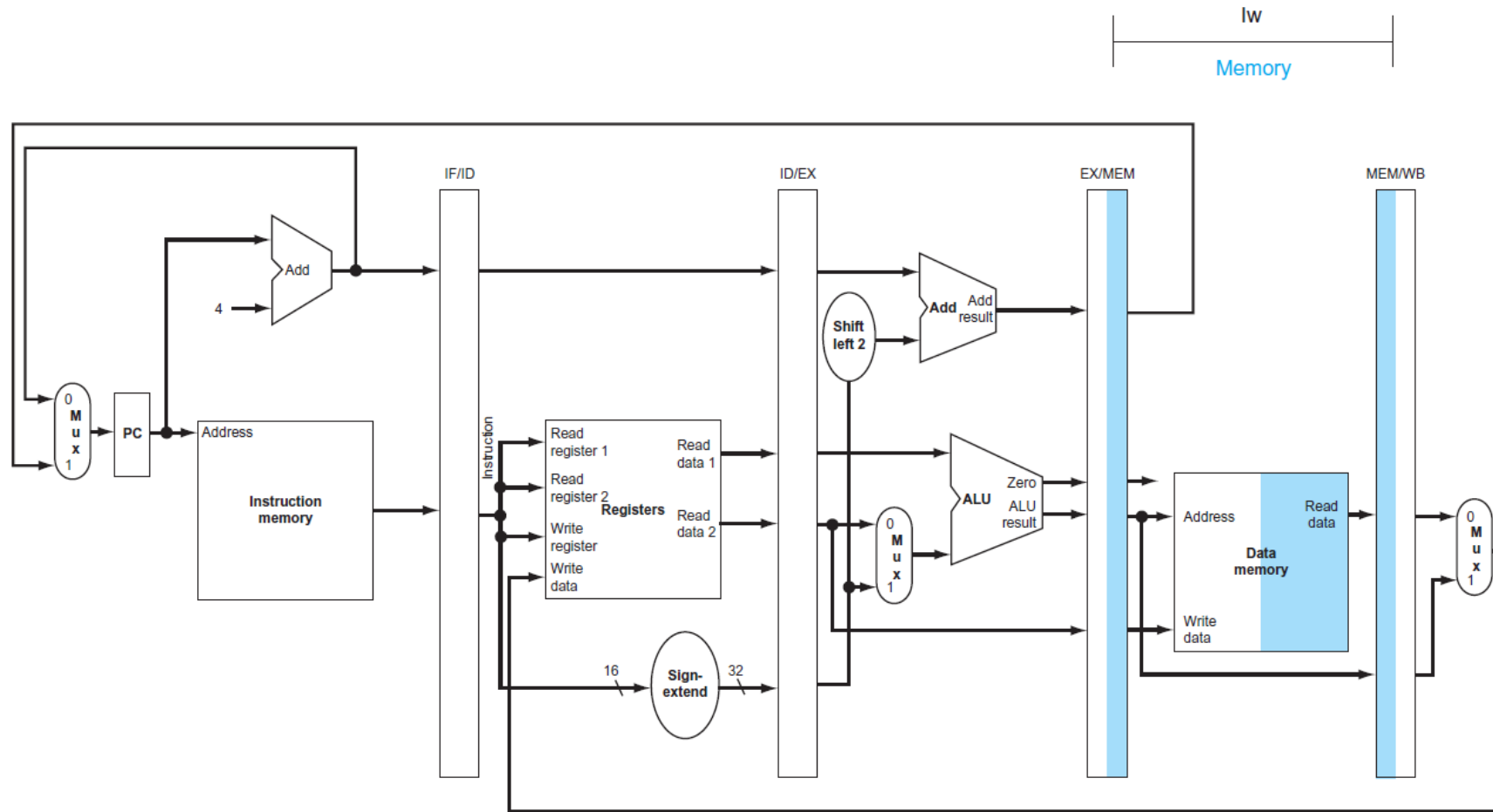
Execute



Memory

- Read data memory using address from EX/MEM pipeline register
- Load data into MEM/WB pipeline register

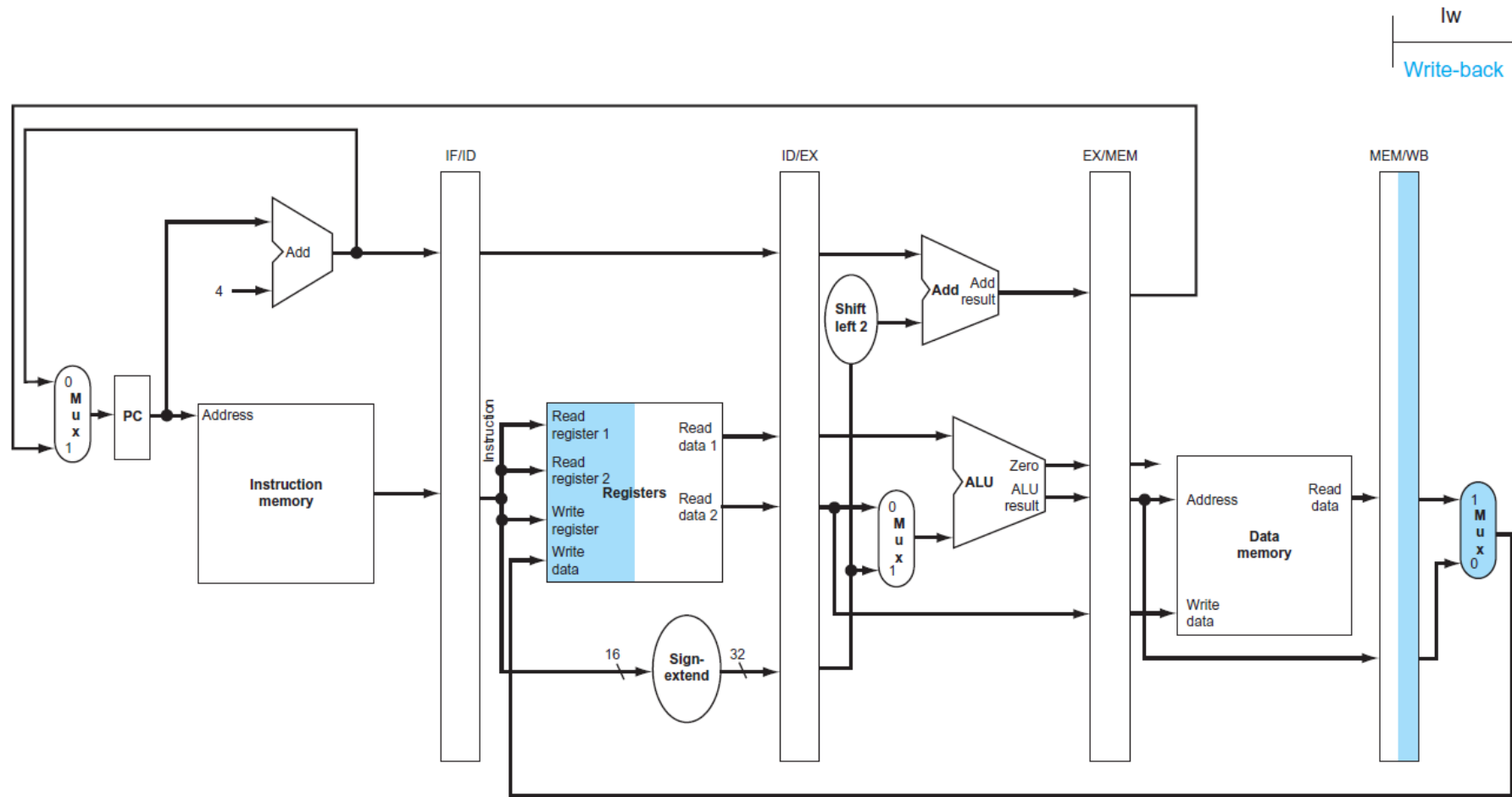
Memory



Write-Back

- Read data from MEM/WB pipeline register
- Write data into register file

Write-Back



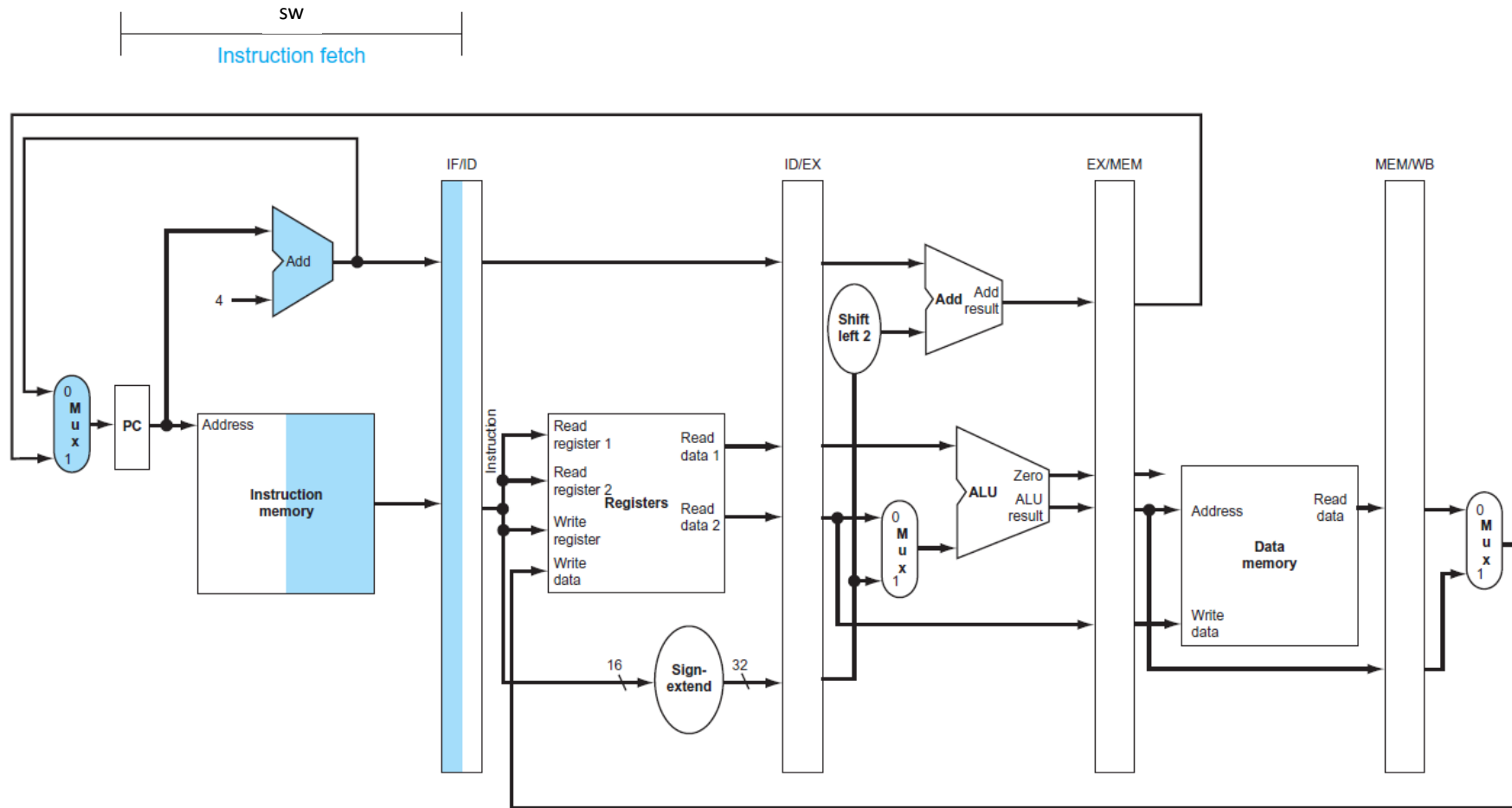
Store-word Instruction Pipeline Datapath

- Walking through store instruction shows similarity of instruction execution, as well as passing information for later stages
- Here are five pipe stages of store instruction

Instruction Fetch

- Instruction is read from memory using address in PC and then is placed in IF/ID pipeline register
- This stage occurs before the instruction is identified

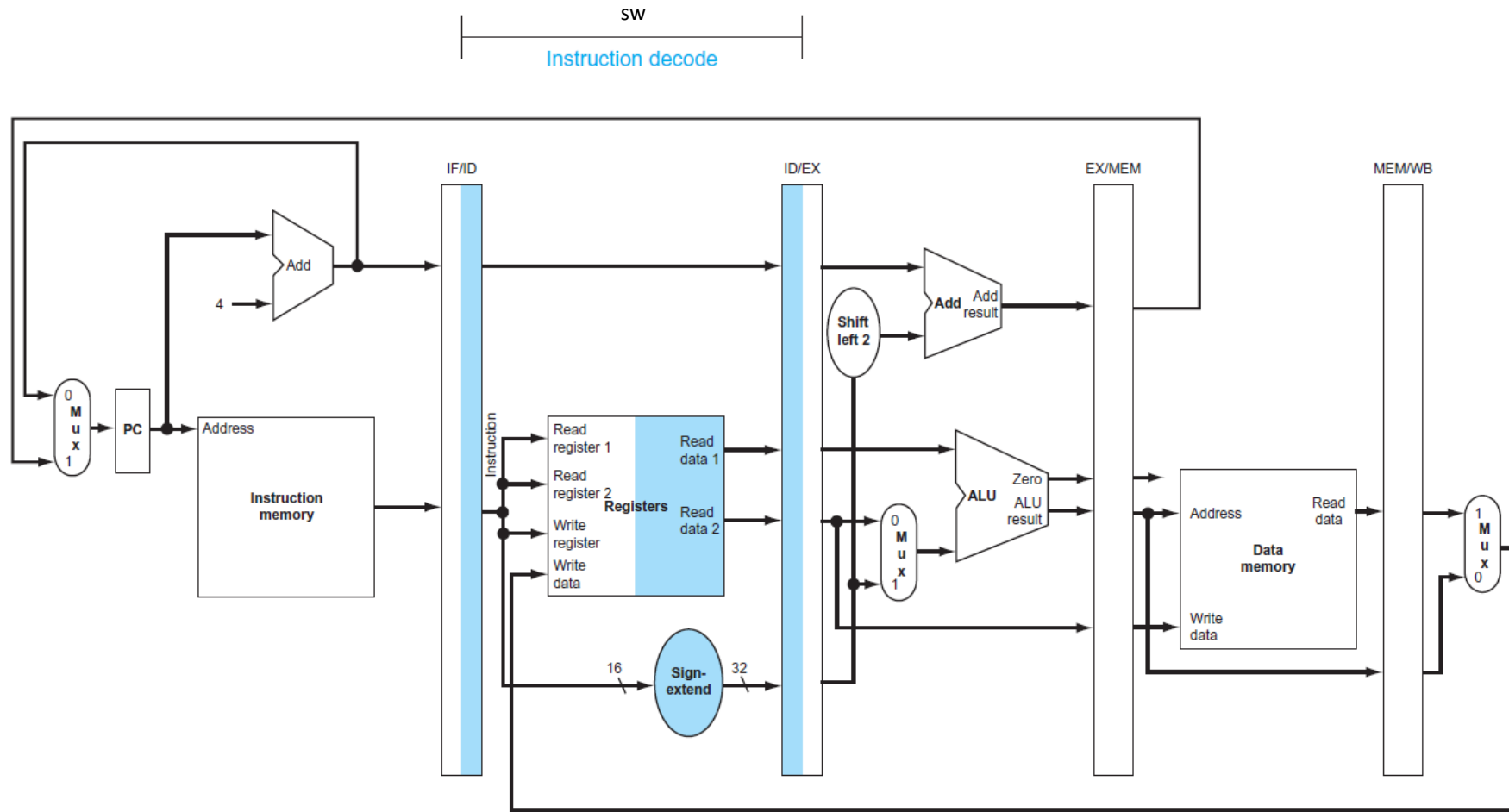
Instruction Fetch



Instruction Decode

- Instruction in IF/ID pipeline register supplies register numbers for reading two registers and extends the sign of the 16-bit immediate
- These three 32-bit values are all stored in ID/EX pipeline register

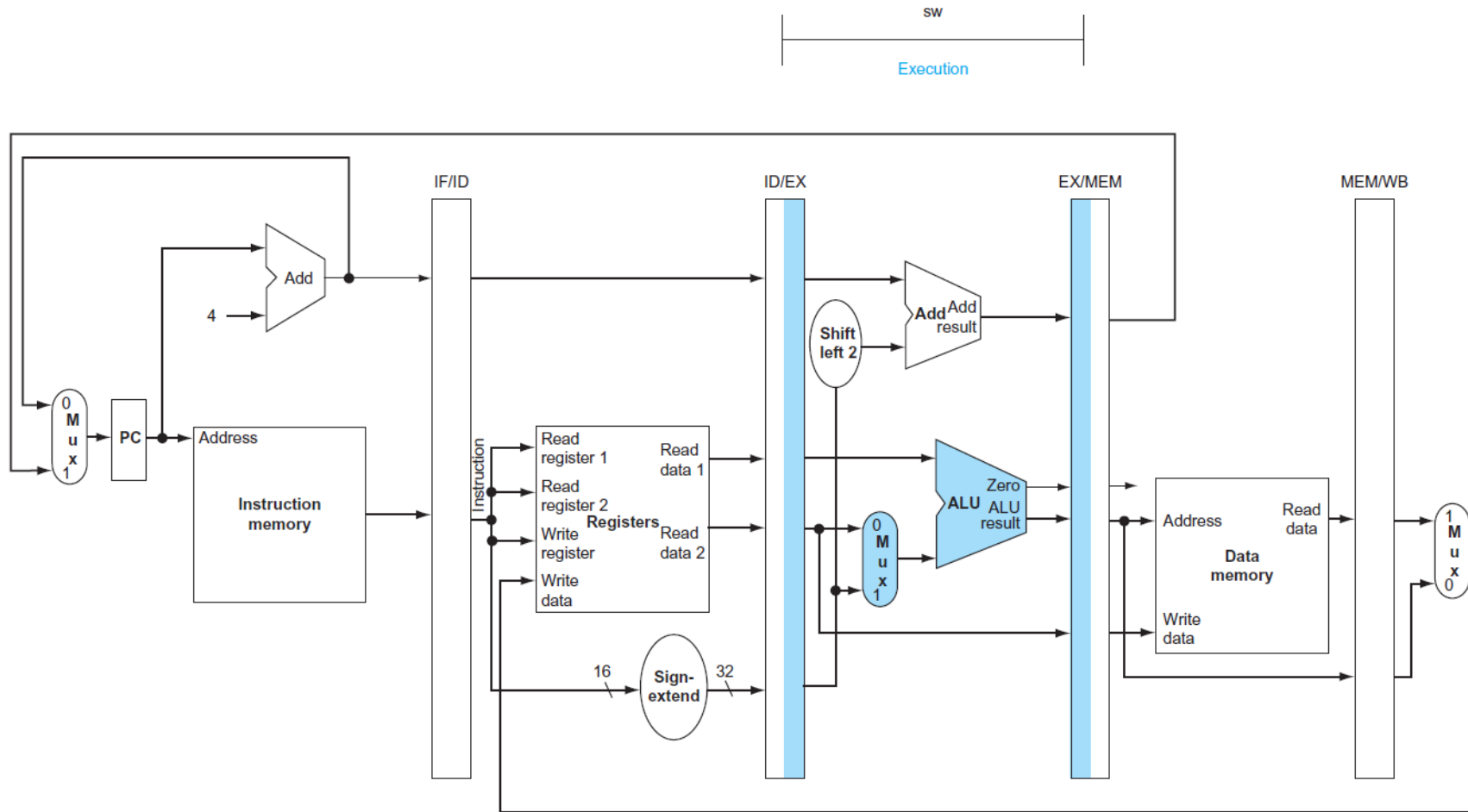
Instruction Decode



Execute

- Effective address is placed in EX/MEM pipeline register

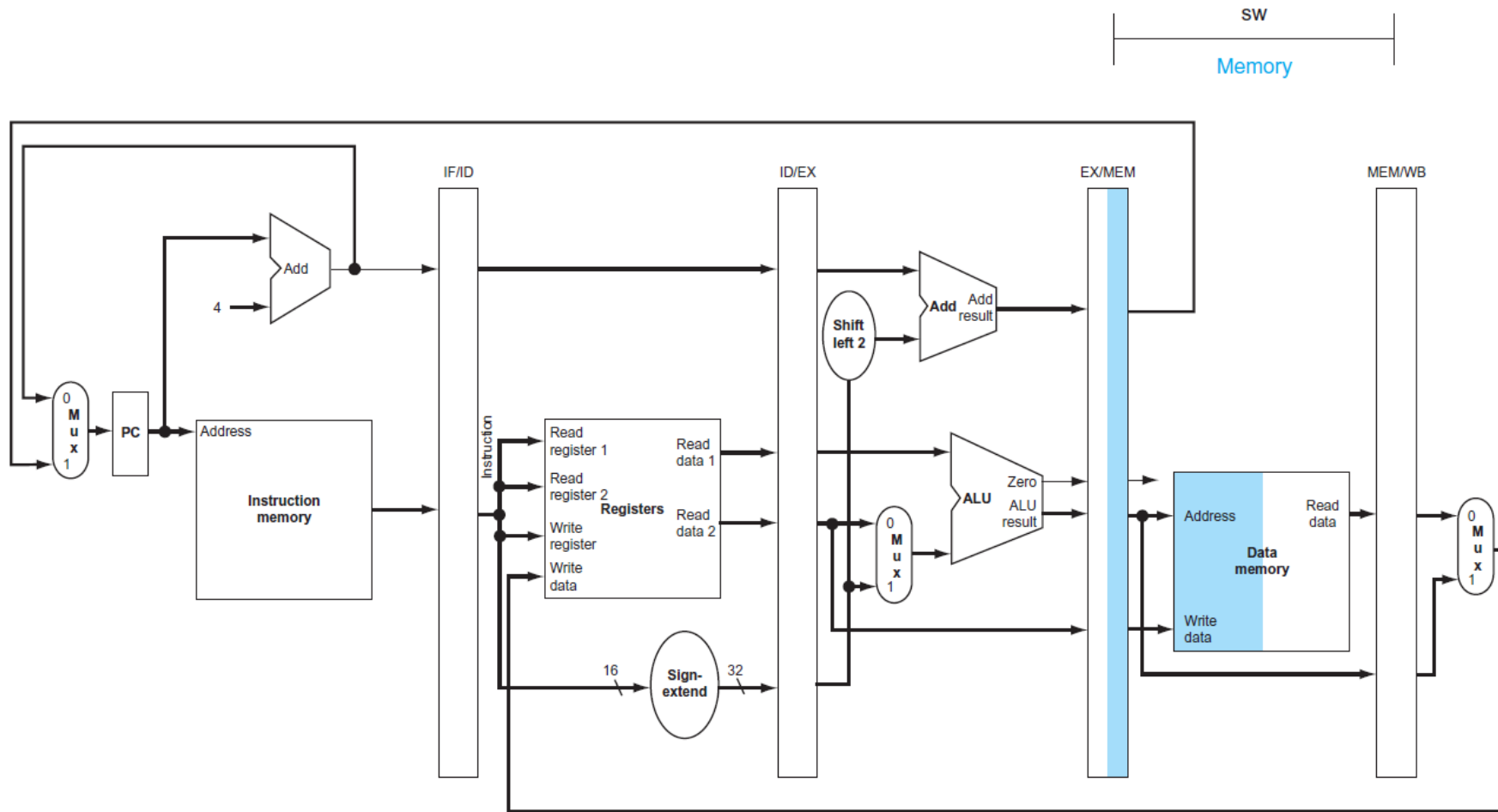
Execute



Memory

- Data is written to memory
- Register containing data to be stored was read in an earlier stage and stored in ID/EX
- The only way to make data available during MEM stage is to place data into EX/MEM pipeline register in EX stage

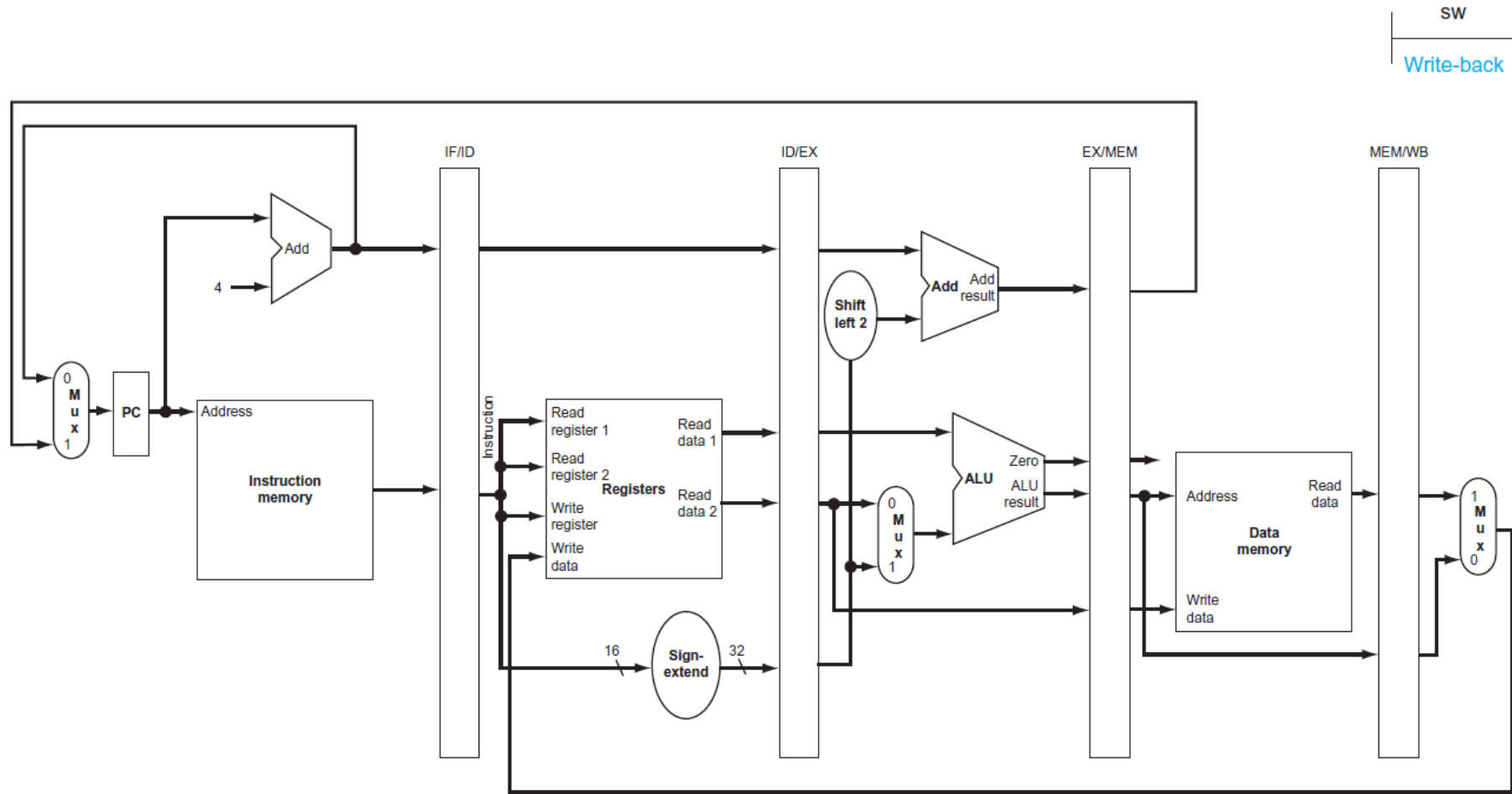
Memory



Write-Back

- For this instruction, nothing happens in the WB stage
- Since every instruction behind store is already in progress, we have no way to accelerate those instructions
- Hence, an instruction passes through a stage even if there is nothing to do, because later instructions are already progressing at the maximum rate

Write-Back



Bug

- Uncover a bug in the design of load instruction

Bug

- Uncover a bug in the design of load instruction
- Which register is changed in the final stage of load?
 - More specifically, which instruction supplies write register number?

Bug

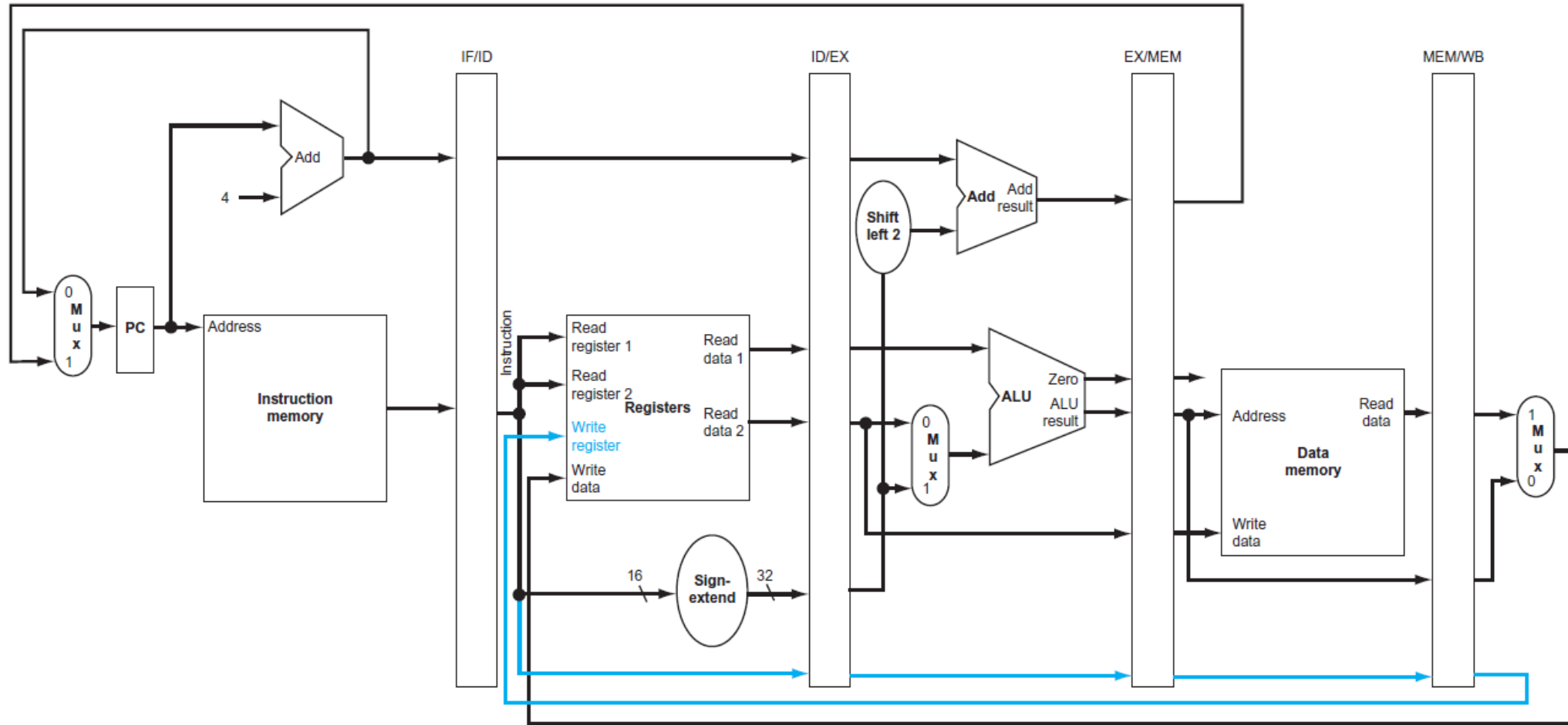
- Uncover a bug in the design of load instruction
- Which register is changed in the final stage of load?
 - More specifically, which instruction supplies write register number?
- Instruction in IF/ID pipeline register supplies write register number
 - Yet this instruction occurs considerably after load instruction!
- Need to preserve destination register number

- Just as store passed register *contents* from ID/EX to EX/MEM pipeline registers for use in MEM stage, load must pass *register* number from ID/EX through EX/MEM to MEM/WB pipeline register for use in WB stage

Corrected Datapath

- Pass write register number first to ID/EX register, then to EX/MEM register, and finally to MEM/WB register
- Register number is used during WB stage to specify register to be written

Corrected Datapath



Datapath of lw Instruction

- Next figure is a single drawing of corrected datapath, highlighting the hardware used in all five stages of the lw instruction

CPCS214, Computer Organization and Architecture, Prepared by Dr. Abdullah Basuhail, CS, FCIT, KAU, 1441H/2020G



Graphically Representation of Pipelines

- In pipelining, many instructions are simultaneously executing in a single datapath in every clock cycle
- To aid understanding, two basic styles of pipeline figures:
 - *Multiple-clock-cycle pipeline diagrams*
 - Simpler but do not contain all details
 - *Single-clock-cycle pipeline diagrams*

Multiple-clock-cycle Pipeline Diagram

- For example, consider the following five-instruction sequence:

lw \$10, 20(\$1)

sub \$11, \$2, \$3

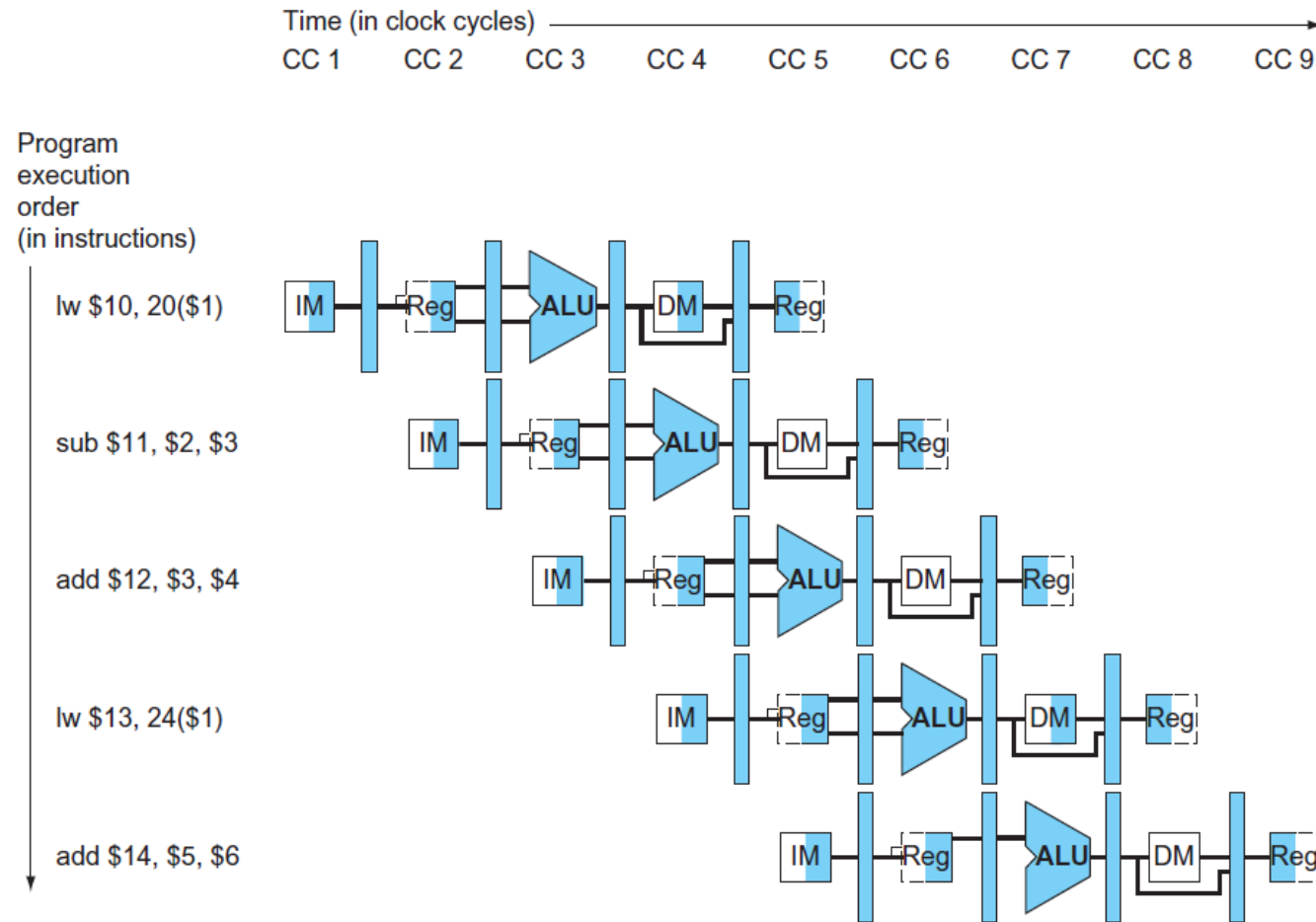
add \$12, \$3, \$4

lw \$13, 24(\$1)

add \$14, \$5, \$6

- Next figure shows multiple-clock-cycle pipeline diagram
 - Time advances LTR; instructions advance TTB

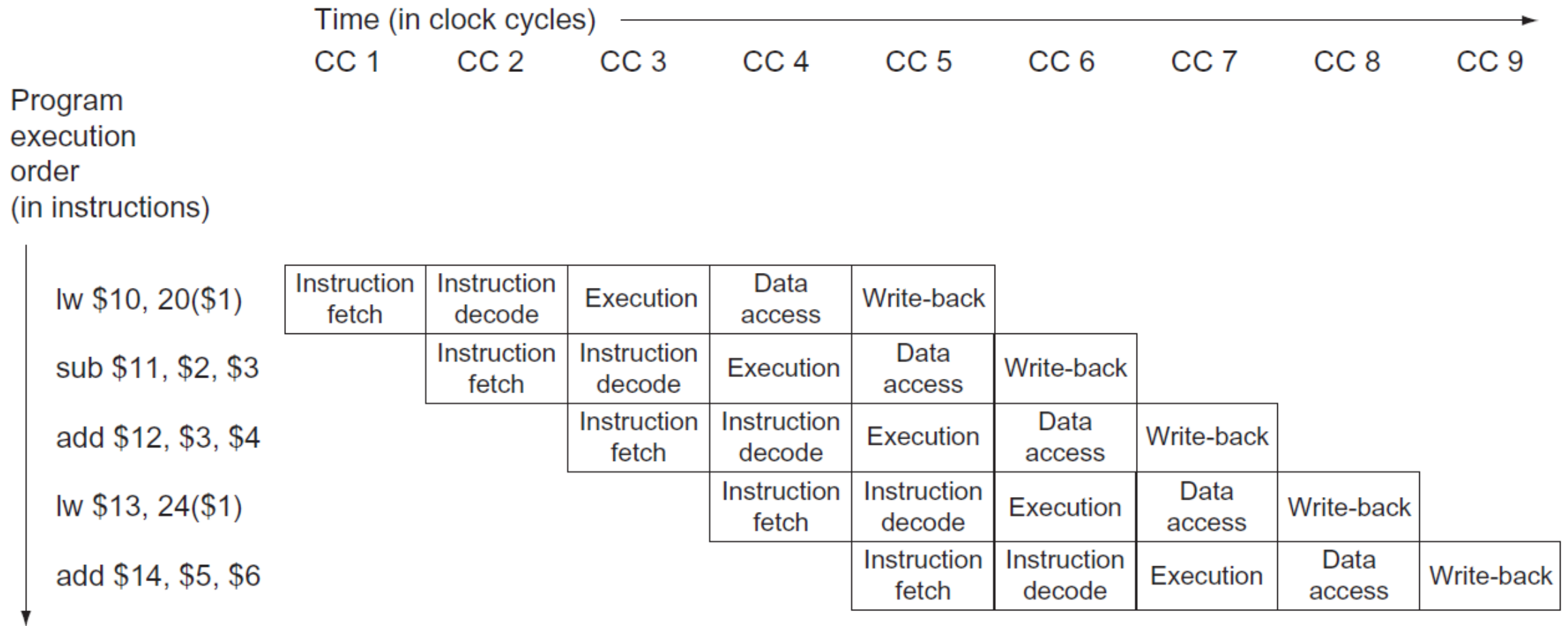
Multiple-clock-cycle Pipeline Diagram



Traditional Pipeline Diagram

- Next figure shows the more traditional version of the multiple-clock-cycle pipeline diagram
- Note that previous figure shows physical resources used at each stage, while next figure uses the *name* of each stage

Traditional Pipeline Diagram

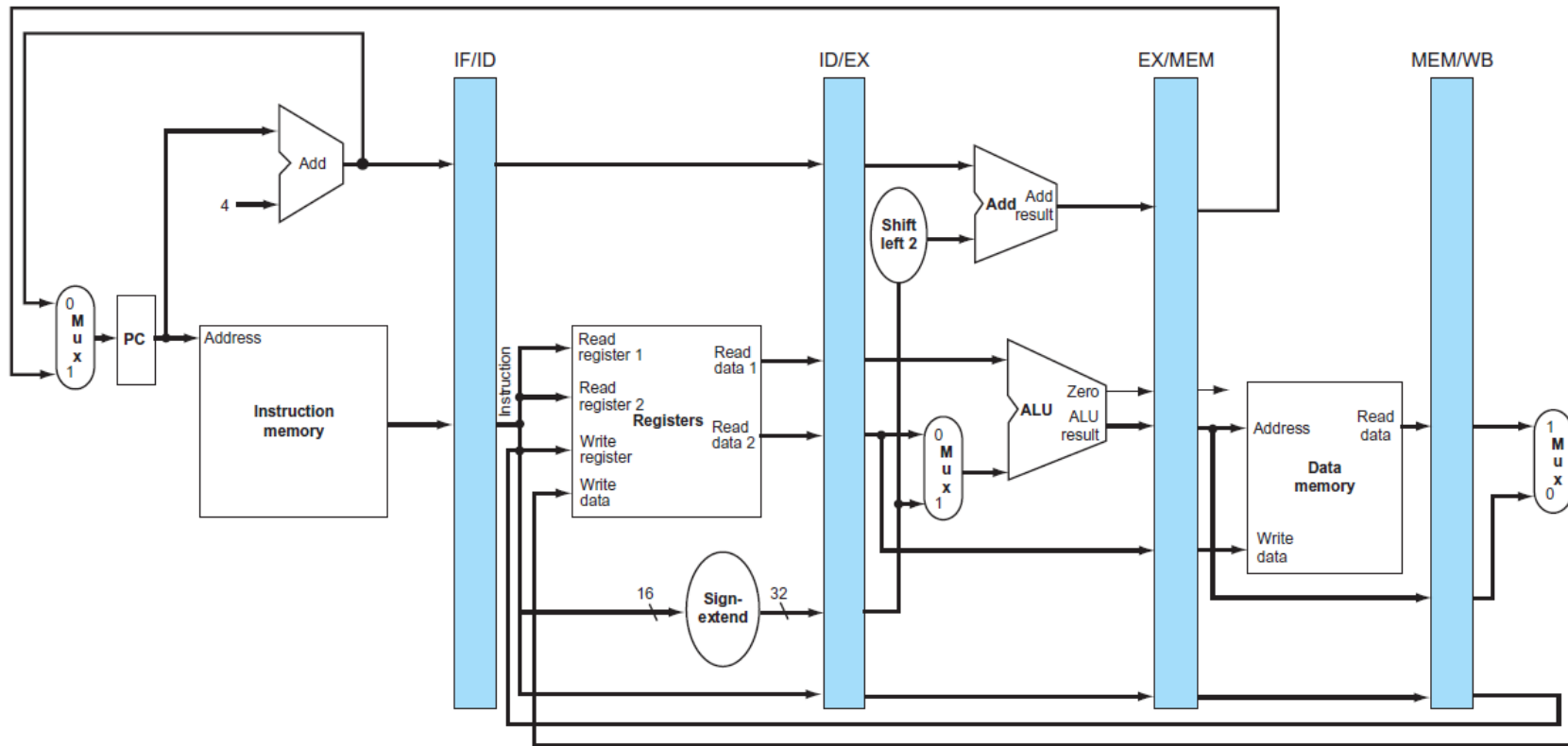


Single-clock-cycle Pipeline Diagram

- Shows state of entire datapath during single clock cycle
- Usually all instructions in pipeline are identified by labels above their respective pipeline stages
- We use this type of figure to show details of what is happening within the pipeline during each clock cycle

Single-clock-cycle Pipeline Diagram

add \$14, \$5, \$6	lw \$13, 24 (\$1)	add \$12, \$3, \$4	sub \$11, \$2, \$3	lw \$10, 20(\$1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back



Pipeline Diagrams

- Use multiple-clock-cycle diagrams to give overviews of pipelining situations
- Single-clock-cycle diagram represents vertical slice through set of multiple-clock-cycle diagrams, showing usage of datapath by each of instructions in pipeline at designated clock cycle